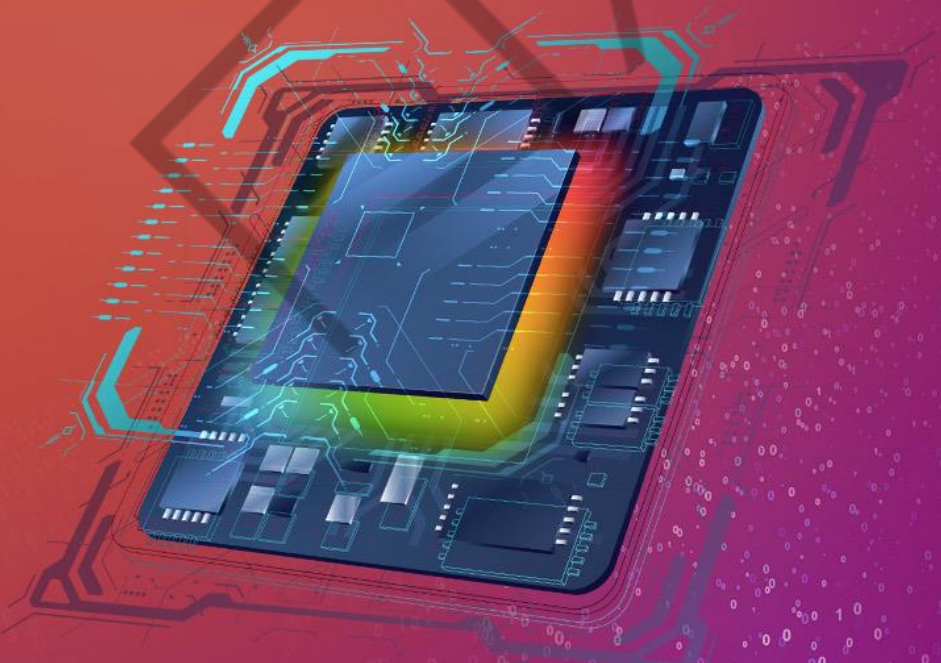


TECH FORCE: GALACTIC
ARCHITECTURE

REGISTRADORES



4

LISTA DE FIGURAS

Figura 1 – Hierarquia de memória.....	7
Figura 2 – Exemplo do registrador EAX.....	10



LISTA DE TABELAS

Tabela 1 – Registradores genéricos IA-32 e sua descrição	11
Tabela 2 – Registradores de segmentos e sua descrição	12
Tabela 3 – Registradores EFLAGS	13

EMENDADO

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplo de notação ATT&T	8
Código-fonte 2 – Exemplo de prefixos	9
Código-fonte 3 – Exemplo de notação IA-32.....	9

EMANIP

SUMÁRIO

REGISTRADORES	6
1 INTRODUÇÃO	6
2 DEFININDO OS REGISTRADORES	6
2.1 Notação ATT&T	8
2.2 Notação Intel IA-32	9
3 REGISTRADORES EM ARQUITETURA X86	10
3.1 General-purpose Registers (GPRs)	10
3.2 Segment Registers	12
3.3 EFLAGS Registers	12
CONCLUSÃO	13
REFERÊNCIAS	14

REGISTRADORES

1 INTRODUÇÃO

Nesse capítulo, iremos mergulhar nas estruturas do processador, explorando os fundamentos dos registradores da arquitetura x86. Essas peças-chave do funcionamento de um computador são verdadeiros guardiões do estado e da manipulação dos dados.

Neste capítulo, examinaremos em detalhes as características e funcionalidades dos registradores x86. Entenderemos como esses elementos são responsáveis por armazenar, manipular e transportar informações cruciais durante a execução dos programas.

Desvendaremos os conceitos fundamentais, tais como registradores de propósito geral (GRPs), registradores de segmentos e registradores de controle. Descobriremos como cada tipo desempenha um papel específico no fluxo das informações e no controle do processador.

Ao final deste capítulo, você estará imerso em um conhecimento profundo sobre os registradores x86, pronto para explorar novos horizontes e utilizar esses elementos de maneira eficiente em suas aplicações. Prepare-se para desvendar os segredos dos registradores e testemunhar o poder e a flexibilidade que eles proporcionam ao processamento de dados.

2 DEFININDO OS REGISTRADORES

Sobre hierarquia de memória, os registradores estão no topo dessa pirâmide.

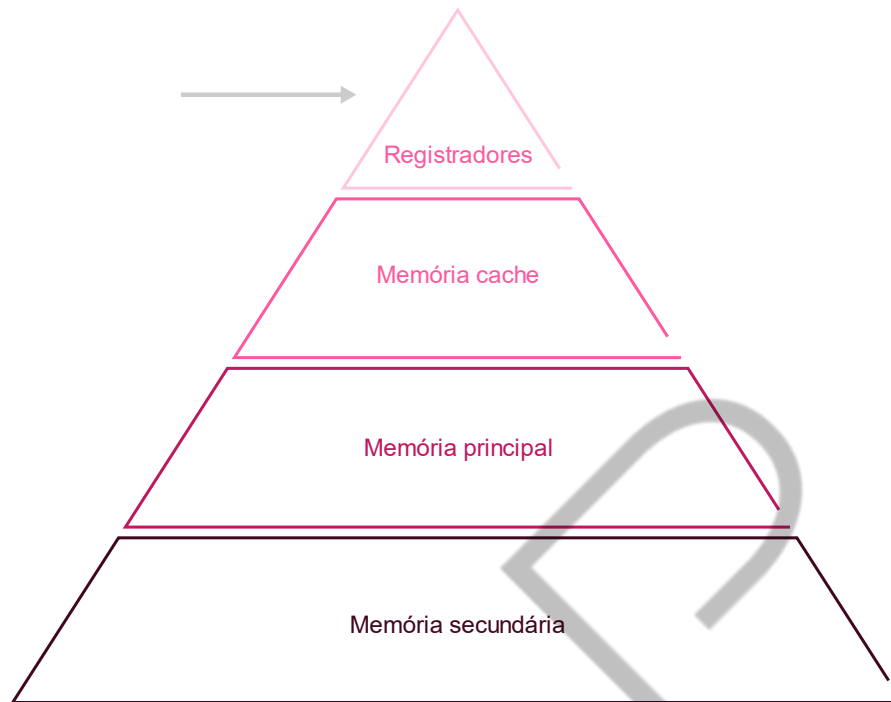


Figura 1 – Hierarquia de memória
Fonte: Elaborada pelo autor (2023)

Esses são pequenos espaços de armazenamento temporário disponível nas CPUs sendo o tipo mais rápido de memória, pois a CPU possui acesso diretamente a ele. Seu tamanho difere dependendo da arquitetura da CPU x86 ou x64, como veremos mais a frente. Utilizando-as para armazenar as instruções de programas em tempo de execução. Existem diferentes categorias de registradores e o seu devido uso, sendo elas:

- **Registradores de uso geral (GPRs):** Esses registradores são usados para armazenar dados e realizar operações lógicas e aritméticas. Eles são usados para manipular valores inteiros, endereços de memória e dados em geral.
- **Registradores de Segmento:** Esses registradores são utilizados para manter o controle da memória.
- **Registradores de Controle:** Esses registradores são responsáveis por controlar o funcionamento do processador.
- **Registradores de Flags:** Também conhecidos como registradores de status, esses registradores contêm bits individuais que indicam o resultado de operações lógicas e aritméticas anteriores.

- **Registradores de ponto flutuante (FPRs):** Esses registradores são projetados para manipular números em formato de ponto flutuante, permitindo a execução de cálculos matemáticos complexos.

Aqui é importante ressaltar que a quantidade e as nomenclaturas dos registradores podem variar dependendo da arquitetura específica e no modo de operação do processador. Existem diversas arquiteturas de CPU e cada uma possui sua notação, as duas mais comuns são a ATT&T e a Intel IA-32.

2.1 Notação ATT&T

A notação ATT&T (também conhecida como sintaxe ATT&T) é uma convenção de formatação utilizada para representar os dados em linguagem Assembly. Essa notação foi desenvolvida pela empresa americana AT&T, que também teve um papel importante no desenvolvimento da arquitetura x86.

Essa notação tem suas raízes no sistema operacional UNIX e é amplamente adotada em sistema baseado em UNIX e Linux. Uma das principais diferenças na Notação ATT&T em relação à Notação Intel (que veremos a seguir) é a ordem dos operandos. Na Notação ATT&T, a origem dos dados é listada antes do destino, enquanto na Notação Intel, a ordem é inversa. Por exemplo, em uma instrução de movimento de dados utilizando a Notação ATT&T teria o seguinte formato:

```
movl %eax, %ebx  
instruction [origem], [destino]
```

Código-fonte 1 – Exemplo de notação ATT&T
Fonte: Elaborado pelo autor (2023)

Nesse simples exemplo, (%eax) representa o registrador de origem e o (%ebx) o registrador de destino. A instrução (movl) é usada para mover um valor de um registrador para outro.

Além disso, a notação ATT&T utiliza uma variante de sufixo para indicar o tamanho dos operadores. Por exemplo, o sufixo (l -Letra L) indica um operando de 32-bit (long), enquanto o (w) representa um operando de 16 bits (word) e (b) representa um operando de 8 bits (byte).

Outras características dessa notação é o uso de prefixo especiais para denotar diferentes tipos de dados, como o (\$) para constantes imediatas e o (%) para registradores. Por exemplo:

addl \$10, %eax

Código-fonte 2 – Exemplo de prefixos
Fonte: Elaborado pelo autor (2023)

Neste caso, a instrução (**addl**) adiciona o valor 10 ao registrador (**%eax**).

Embora a Notação ATT&T possa parecer um pouco confusa assim à primeira vista. Isso depende muito da escolha entre ATT&T e IA-32 elas geralmente dependem da plataforma ou ambiente de desenvolvimento específico e das preferências pessoais dos programadores.

2.2 Notação Intal IA-32

Essa é a notação mais comum que você irá encontrar em diferentes recursos e será com ela que iremos trabalhar ao longo da matéria.

Essa notação também utiliza um conjunto de instruções codificadas em linguagem de máquina que são executadas diretamente pelo processador. Diferente da notação ATT&T, a IA-32 é uma notação mais limpa e sem muito detalhes como caracteres como (% , \$). No exemplo abaixo podemos ver um exemplo da notação IA-32.

mov eax, 5

mov ebx, 3

add ecx, eax

add ecx, ebx

Código-fonte 3 – Exemplo de notação IA-32
Fonte: Elaborado pelo autor (2023)

Nesse exemplo, as instruções (mov) são utilizadas para mover os valores para os registradores (EAX e EBX). Em seguida utilizamos as instruções (add) para adicionar (ou somar) os valores dos registradores (EAX e EBX) armazenando esse resultado no registrador (ECX).

No entanto, é importante ressaltar que a programação em IA-32 é uma abordagem de baixo nível e requer um conhecimento profundo do hardware subjacente. Em geral, a maioria dos programadores modernos preferem usar linguagens de programação de alto nível, como C, C++ ou Java e confiar ao compilador para traduzir o código para a notação IA-32 de forma eficiente.

3 REGISTRADORES EM ARQUITETURA X86

Agora vamos entender para que cada um dos registradores serve tanto na arquitetura x86 e tanto na x64.

Como mencionamos anteriormente, cada um dos registradores possuem uma funcionalidade específica devido a sua utilização. Arquitetura x86 consegue armazenar valores até 32 bits, ou seja, esse valor vai de **0x00000000** até **0xFFFFFFFF**. Já arquitetura x64 conseguem armazenar valores de **0x0000000000000000** até **0xFFFFFFFFFFFFFFFF**.

Vamos ver cada um desses registradores e a suas utilidades.

3.1 General-purpose Registers (GPRs)

Os registradores gerais são normalmente usados para manipular e armazenar dados, endereços de memórias, e são constantemente utilizados de forma que se intercalam entre si para conseguirem processar corretamente o programa.

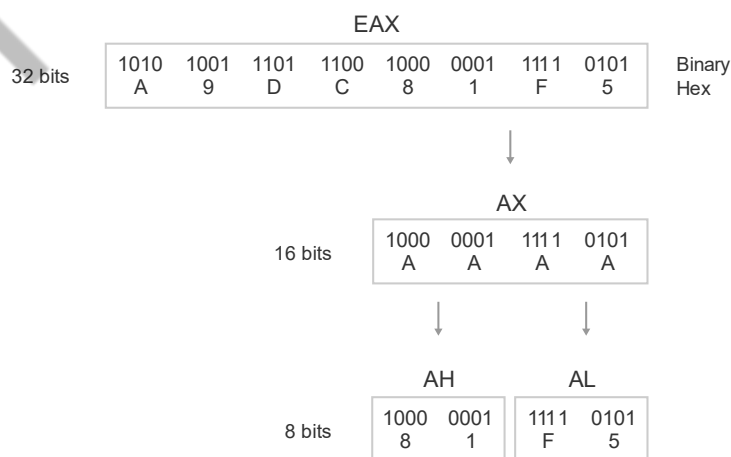


Figura 2 – Exemplo do registrador EAX
Fonte: SIKORSKI; HONIG (2012)

Na Figura “Exemplo do registrador EAX”, vemos uma representação dos nomes e dos tamanhos que um dado pode ser armazenado no em cada registrador na arquitetura x86. Os registradores são divididos da seguinte maneira. A versão de 8 bits suporta até 1 bytes dividido como AH (Accumulator High) e AL (Accumulator Low). A versão de 16 bits AX (Accumulator) consegue armazenar dados menores em tamanho, como valores inteiros de 16 bits, ou seja, 2 bytes. Já a versão de 32 bits (Extended Accumulator) permite armazenarmos 4 bytes.

Registradores gerais	Descrição
EAX (AX, AH, AL)	O registrador EAX é utilizado para armazenar valores temporários, resultados de cálculos e retornos de funções. É o registrador mais utilizado para operações aritméticas e lógicas. Também é usado para armazenar valores de 32 bits e é especialmente útil em operações de multiplicação e divisão.
EBX (BX, BH, BL)	O registrador EBX é geralmente utilizado como um registrador de propósito geral para armazenar valores, endereços de memória e ponteiros. Ele também pode ser usado para operações aritméticas e lógicas, assim como o EAX.
ECX (CX, CH, CL)	O registrador ECX é comumente utilizado em laços e iterações, como contador. Ele também é frequentemente utilizado para controlar loops e realizar contagens, diminuindo o valor armazenado até atingir zero. Também pode ser utilizado como um registrador de propósito geral quando não está sendo usado para controle de loops.
EDX (DX, DH, DL)	O registrador EDX é geralmente utilizado em operações de multiplicação e divisão de números maiores. Ele auxilia o registrador EAX no armazenamento de resultados dessas operações. Além disso, o EDX é frequentemente usado para armazenar valores de 32 bits que não podem ser acomodados no EAX.
EBP (BP)	É frequentemente utilizado como um ponteiro de base para acessar os parâmetros e variáveis locais em sub-rotinas e funções.
ESP (SP)	É o ponteiro de pilha (stack pointer) e é usado para controlar a pilha de chamadas de sub-rotinas e gerenciar o espaço de memória temporário durante a execução do programa.
ESI (SI)	Usado como índice de origem em operações de transferência de dados e também é usado para percorrer elementos em estruturas de dados.
EDI (DI)	Usado como índice de destino em operações de transferência de dados e também é usado para percorrer elementos em estruturas de dados.

Tabela 1 – Registradores genéricos IA-32 e sua descrição
Fonte: Elaborada pelo autor (2023)

Esses registradores são apenas algumas das opções disponíveis e sua utilização específica pode variar dependendo do contexto do programa. Cada registrador tem sua função especializada, mas também podem ser utilizados de forma flexível para diferentes propósitos durante a execução do código.

3.2 Segment Registers

Registradores de segmentos como (CS, DS, SS, ES, FS e GS) são registradores especiais encontrados em arquiteturas de x86. Eles são usados para controlar a forma como a memória é acessada e organizada. Na tabela abaixo podemos ver uma explicação para cada um desses registradores:

Registrador de segmento	Descrição
CS (Code Segment)	Este registrador armazena o segmento de código atual. Ele indica ao processador onde buscar as instruções que serão executadas. Ele é usado principalmente para controlar o fluxo de execução do programa.
DS (Data Segment)	O registrador DS contém o segmento de dados atual. Ele é usado para acessar dados na memória, como variáveis e estruturas de dados.
SS (Stack Segment)	O registrador SS aponta para o segmento de pilha atual. A pilha é uma área especial da memória usada para armazenar informações temporárias, como endereços de retorno de chamadas de função e variáveis locais. O SS controla a pilha do programa.
ES (Extra Segment)FS (Additional Segment)GS (Additional Segment)	Os registradores (ES, FS, GS) são um segmento adicional usado para acessar dados extras na memória. Ele é usado para operações que requerem acesso a segmentos de dados adicionais além do segmento de dados padrão (DS).

Tabela 2 – Registradores de segmentos e sua descrição
Fonte: Elaborada pelo autor (2023)

3.3 EFLAGS Registers

Os registradores EFLAGS, também são conhecidos como Flags de status, esses são um conjunto de bits de processadores x86 que registram o resultado de operações aritméticas e lógicas. Eles fornecem informações sobre o estado atual do processador, permitindo que o sistema operacional e os programas monitorem e controlem o fluxo de exceção.

Os registradores EFLAGS são compostos por vários bits, cada um com um significado específico. Aqui estão os principais bits e suas funções:

Registradores EFLAGS	Descrição
CF (Carry Flag)	O Carry Flag indica se houve um transporte de bit (carry) a partir da operação aritmética mais significativa. Por exemplo, quando uma adição resulta em um valor maior do que o espaço disponível para armazenamento, o CF é definido para indicar o transporte.
PF (Parity Flag)	O Parity Flag indica a paridade (número de bits 1) do resultado de uma operação. Se o número de bits 1 for par, o PF é definido como 1; caso contrário, é definido como 0.
AF (Auxiliary Carry)	O Auxiliary Carry Flag é usado para indicar um transporte entre os bits 3 e 4 durante operações aritméticas em BCD (Binary-Coded Decimal). Geralmente, é utilizado para operações com dígitos decimais.
ZF (Zero Flag)	O Zero Flag indica se o resultado de uma operação é zero. Se o resultado for zero, o ZF é definido como 1; caso contrário, é definido como 0.
SF (Sign Flag)	O Sign Flag indica o sinal do resultado de uma operação. Se o resultado for negativo, o SF é definido como 1; caso contrário, é definido como 0.
OF (Overflow Flag)	O Overflow Flag é usado para detectar quando ocorre um estouro de bit durante uma operação aritmética, indicando que o resultado não pode ser representado corretamente com o tamanho dos operandos.

Tabela 3 –Registradores EFLAGS
Fonte: Elaborada pelo autor (2023)

Em resumo, os registradores EFLAGS fornecem informações sobre o estado atual do processador, permitindo que os programas e o sistema operacional tomem decisões com base nesses sinais. Eles desempenham um papel fundamental no controle do fluxo de execução e no tratamento de exceções em sistemas x86.

CONCLUSÃO

Desde os registradores de uso geral (GPRs), que são os verdadeiros protagonistas das operações computacionais, até os registradores de segmentos e os EFLAGS, responsáveis pelo controle e estado do processamento, cada um desempenha um papel crucial na execução de instruções e na interação com a memória. Ao compreendermos a importância e as funcionalidades desses registros, adquirimos um conhecimento fundamental para explorar o vasto universo da arquitetura x86, permitindo-nos desbravar novos horizontes tecnológicos e alcançar feitos surpreendentes no campo da computação.

REFERÊNCIAS

INTEL. Intel 64 and IA-32 Architectures Software Developer's Manual. s.d. Disponível em: <<https://software.intel.com/content/www/us/en/develop/articles/intel-sdm.html>>. Acesso em: 19 jun. 2023.

SIKORSKI, Michael; HONIG, Andrew. **Practical Malware Analysis**: The Hands-on Guide to Dissecting Malicious Software. San Francisco, USA: No Starch Press, 2012.

EMANIP